RHIDZKHAN AHMAD

Technical AI Lead / Solution Architect

+63 927 457 7680 | rhidzkhan.ahmad99@gmail.com | linkedin.com/in/rhidzkhanahmad
github.com/cokhane | mehruemm99.pythonanywhere.com | Design portfolio: youtube.com/watch?v=phL5W-biJWU

PROFESSIONAL SUMMARY

Technical AI Lead and Solution Architect with 10+ years designing Python and Java API platforms and the multi-cloud infrastructure beneath them. Currently lead engineering on a pharma competitive-intelligence SaaS running on containerised AWS Lambda, MongoDB, and the Anthropic Claude API, architected an in-house Zoom-class video interview platform from scratch, and build the AI agents and skills the engineering org develops with. Previously built the payment and subscription systems behind 1,000+ orders/day, and scaled an MSP platform from 1 to 10 clients while cutting infrastructure cost 67%. Employee of the Year, Visibility.

TECHNICAL SKILLS

Backend: Python (FastAPI, Django, Flask, Tornado), Java (Spring Boot), Node.js, TypeScript, REST APIs, GraphQL, microservices, WebSocket, MedusaJS

Cloud & DevOps: AWS (Lambda, ECS, EC2, RDS, ElastiCache, ALB, EKS, S3, CloudFront, EventBridge, Kinesis, Glue, Athena, Redshift, DMS, KMS, Macie, GuardDuty), GCP, Azure, Docker, Terraform, CloudFormation, Ansible, Jenkins, Continuous Integration/Continuous Deployment (CI/CD)

Data & Analytics: PostgreSQL, MySQL, MongoDB, Redis, DynamoDB, complex SQL, stored procedures, query optimization, PySpark, Databricks, Medallion (Bronze/Silver/Gold) architecture, Azure Data Factory, Synapse, Event Hub, ETL pipelines, Celery, RabbitMQ

Observability: Prometheus, Grafana, CloudWatch, Azure Monitor, Log Analytics, Application Insights, Microsoft Sentinel (SIEM), alerting pipelines

Architecture: Multi-tenant cell/stamp design, multi-AZ high availability, Azure Landing Zone (Microsoft CAF), payment orchestration, Infrastructure as Code (IaC), LLM integration

Testing & Frontend: pytest, Selenium, unit/integration/end-to-end testing, Test-Driven Development (TDD), React, Next.js, Vue.js, Angular

PROFESSIONAL EXPERIENCE

ANERVEA.AI — Technical AI Lead / Solution Architect — May 2026 – Present

Jira Delivery Intelligence Tracker — multi-board engineering delivery, developer-KPI and capacity platform

Java 21 · Spring Boot 3 · Spring Web MVC · Spring Data JPA / Hibernate · HikariCP · Flyway · SQLite per-tenant (PostgreSQL-ready) · Quartz Scheduler · Spring RestClient · Jackson · Spring AI / Anthropic · Apache POI (XSLF + XSSF) · springdoc-openapi · Spring Boot Actuator · React + Vite · Recharts · Maven · JUnit 5 + Testcontainers

- Architected a self-hosted delivery-intelligence service that pulls several Jira boards into per-board time-series databases on a nightly schedule, manufactures the day-by-day board history Jira does not expose, and serves sprint velocity, per-developer KPI, capacity planning, movement and QA reporting to a React operator console — plus Markdown, PPTX and printable-HTML deliverables — under a hard constraint of read-only against Jira and zero change to how the team already works, so adoption costs the engineers nothing
- Designed board-isolated persistence as true database-per-tenant multi-tenancy — per-board HikariCP pools behind a MultiTenantConnectionProvider with the tenant resolved from the request path, so an unknown board returns 404 before a connection is ever leased and cross-board contamination is structurally impossible rather than a WHERE clause someone can forget; repo-scoped pull-request data gets its own DataSource as an argued exception

- Built the time machine Jira does not expose — an append-only daily snapshot keyed on (snapshotDate, issueKey) giving exactly one immutable row per ticket per day, with a reconciler holding measured history strictly senior to changelog-reconstructed history so no day is ever half-measured and half-inferred
- Made developer attribution an explicit, defended policy rather than an implementation detail — delivery credits the roster developer who moved the ticket, QA send-backs credit the owner at the time of the move, and a monitored-but-not-rostered distinction stops the lead account absorbing others' work: a correction worth 328 of ~460 tickets on one board and 188 of 211 on the other
- Drew a hard determinism boundary around the LLM — every number, table and funnel renders from the database and the model is confined to prose, with a missing API key returning a typed 503 rather than letting it improvise a figure; the developer KPI is likewise built to resist the behaviour it would create, capping delivery at 100% of planned capacity and scoring quality zero on zero delivery
- Built negative-space monitoring into the scheduler — a dashboard of the rows that exist shows green while the scheduler is dead, so an Actuator HealthIndicator replays the Quartz CronExpression to reconstruct the nights the job was expected to fire, separating missed, unscheduled and failed; typed @ConfigurationProperties refreshed via @RefreshScope likewise make onboarding a board a config block and a rotated token need no restart

AI Interviewer — real-time AI avatar interview platform for physicians and KOLs

Python 3.11 (aiohttp) · React 19 + TypeScript · AWS ECS Fargate/ALB · Terraform · Redis · MongoDB · Anthropic Claude (Opus 5 + Haiku 4.5)

- Designed a real-time, event-driven conversational AI platform conducting live video interviews with clinicians — Python 3.11/aiohttp backend, React 19 + TypeScript front end, deployed as containerised services on AWS ECS Fargate behind an ALB, provisioned entirely in Terraform
- Structured the backend as a hexagonal modular monolith — a pure, I/O-free domain core behind thin vendor adapters and a tested composition root — with module purity enforced mechanically in CI rather than by convention
- Architected stateless session handling with Redis as the system of record, distributed per-session locking and reconnect rehydration, so a mid-interview process restart never drops a live physician session
- Eliminated all WebRTC ownership by keeping the media leg browser-to-vendor while relaying audio server-side — delivering recording, credential isolation and barge-in authority with no SFU or TURN infrastructure
- Specified a two-tier LLM topology (Claude Opus 5 offline for guide synthesis, Haiku 4.5 online for constrained per-turn selection) that holds the agent to pre-approved clinical content, against a measured 975 ms p95 avatar budget and 1.4–2.4 s end-to-end turn latency
- Replaced an absent vector-search capability with a three-stage MongoDB retrieval funnel, distilling 21,832 raw signals into 13 grounded interview questions at $0.13 per guide, with ungrounded questions rejected mechanically
- Embedded regulatory compliance in the architecture: a single screened egress path for all synthesised speech, pre-model pseudonymisation, and consent enforced by construction — the recording sink and upload presign do not exist before consent
- Resolved cross-team AWS ownership with a lifecycle-plus-singleton rule and an SSM Parameter Store contract between estates, failing loudly at plan time instead of silently at apply
- Delivered under a zero-suppression quality gate — ruff, flake8, pylint 10.00/10, 1,864 tests at 100% enforced coverage — across ~60 tickets in 8 phases with full decision-to-ticket traceability

Autonomous Claude Bot — engineering agent platform

Headless Claude Code runners · Python · Jira REST API · GitHub (Actions, CODEOWNERS) · Terraform

- Designed and shipped the Autonomous Claude Bot, a production coding agent that works Jira tickets end-to-end — queue selection, repo routing, headless implementation, signed commits, PR — within a 30-runner fleet spanning 10 Jira projects and ~15 repositories

- Extended the runner into a DevOps agent with ownership of CI workflows and Terraform under a deliberate plan-only policy (writes infrastructure code, never applies it), separating authorship from deployment authority
- Authored the guardrail model for unattended agents holding live credentials: a prompt-level operating charter, forbidden-path push hooks, and queue scoping by assignee — including blocking CODEOWNERS in all three locations GitHub reads it, so an agent cannot select its own reviewers
- Contributed to the organisation's shared agent governance layer (134 skills, 13 subagents, 10 cross-cutting engineering rules) vendored into every runner — covering code review gates, PR stacking, promotion-lane direction, and Jira write verification
- Established the platform invariant that every agent PR passes a two-agent quality gate and a human merge — no autonomous merge path exists

Nexus — real-time pharma competitive-intelligence SaaS

Python 3.11 · FastAPI · AWS (Lambda container images, ECS Fargate, EventBridge, SQS, S3, ECR, Secrets Manager, KMS, VPC/PrivateLink) · MongoDB Atlas · PostgreSQL · Redis · Neo4j / Amazon Neptune · Terraform · GitHub Actions · Anthropic Claude API · LightGBM

- Architected a two-layer multi-tenant platform separating a shared data plane from a per-tenant read lens: intelligence is collected once per disease indication into a common lake, then personalized per customer at read time — so collection cost scales with indications rather than customers, and no tenant-specific copy of a record exists anywhere in the system
- Applied database-per-tenant isolation for all client-private state (decisions, dismissals, vault, reports) against a shared indication-keyed lake for everything else, making cross-tenant leakage a structural impossibility rather than a query-filter convention
- Designed an event-driven fan-out topology with bulkhead isolation — a dispatcher Lambda launching one independently containerized worker per source on EventBridge schedules, 100+ concurrent, with dead-letter queues — so a single source's rate-limit, outage or hard block cannot stall the remaining 121
- Built the ingestion path as a staged pipes-and-filters pipeline — collect → normalize to a canonical envelope → resolve and tag entities → score → compound — over a tiered object lake (raw → normalized → Parquet archive), with content-hash deduplication and change detection as the idempotency boundary between stages
- Introduced CQRS-style materialized read models, with background materializers precomputing the landscape, product and graph projections so the customer-facing read path serves ranked views without recomputing the pipeline
- Designed a compute-once / personalize-at-read scoring model — a universal relevance boost written at ingestion, multiplied by per-tenant weights at request time — which is what lets one immutable shared record produce a different ranking for every customer
- Applied ports-and-adapters to the volatile dependencies: one graph interface with interchangeable Neo4j (Bolt) and Neptune (openCypher) backends, and a chain-of-responsibility enrichment fallback over tiered Claude model routing with per-call cost accounting
- Codified the whole estate as Terraform across four workspaces on a two-tier shared-resource model (dev+stg, uat+prod) with OIDC-authenticated deploys and a generated image registry driving per-service container builds, making the blast radius of a shared-layer change an explicit design constraint

Nexus Data QC Agent — AI-assisted data quality control microservice for a clinical data platform

Python · FastAPI (async) · React · MongoDB · PostgreSQL · Redis · AWS S3 · Anthropic Claude · Docker Compose · nginx · Jenkins

- Architected a standalone QC microservice that reconciles pharma collector output against the curated, client-served database, auto-corrects provably-safe defects and escalates the rest to human review — Python/FastAPI async backend over MongoDB, PostgreSQL, Redis and S3, React operator console, containerised on Docker Compose behind nginx with a test-gated Jenkins pipeline, under a hard constraint of zero modification to the host platform's codebase or schema

- Designed a trust-tiered persistence topology — three separately-credentialed MongoDB connections (read-only collector store, write-gated curated store, service-owned state store) and two S3 buckets split the same way — so read-only guarantees are enforceable at the database-grant and IAM level rather than in application code alone
- Established structural enforcement over documented convention as the governing principle: read-only repositories define no write methods, the blob store cannot express an out-of-scope key, and a 10-rule architectural safety contract is cited in the docstring of each enforcing module and asserted by dedicated CI tests — making unsafe operations unrepresentable rather than merely discouraged
- Specified a single-chokepoint write path where every mutation to the curated database converges on one function, guaranteeing that field precedence, conflict detection, vendored validation guards, change history, lineage and downstream cache invalidation run on every write without exception
- Designed a cascading QC pipeline running the cheapest deterministic tiers first — payload-only content hashing, schema conformance gating, then cross-database reconciliation — so the model evaluates only the residual deterministic logic could not settle, a simultaneous cost and quality control

AlfaKinetic — multi-tenant client API over the Nexus pipeline

AWS Lambda (one container image per endpoint) · API Gateway HTTP API · MongoDB/DocumentDB · PostgreSQL · Redis · EventBridge Scheduler

- Own the client tier serving four live pharma tenants — 265 container-image Lambda functions, one per endpoint, reaching Nexus only through an HTTP client so the client tier never touches the pipeline's datastore
- Lead the migration of a pharma client off a single-tenant competitive-intelligence backend onto the multi-tenant platform: mapped all 233 legacy operations across 43 domains, classified each as live, backlogged, superseded, or unbuilt, and sequenced the remainder into four delivery phases

AlfaKinetic V2 — multi-tenant pharmaceutical competitive-intelligence SaaS on AWS

Architecture: Serverless function-per-endpoint on AWS Lambda — 308 independently deployable container-image functions plus WebSocket and scheduled-worker lanes, fanned out from committed JSON registries through a single reusable Terraform module

System design: CQRS with materialized per-tenant read models · event-driven asynchronous pipelines over SQS with exponential-backoff retry and DLQ · centralized API Gateway authorizer · database-per-tenant isolation with shared-data/client-overlay scoping · Postgres-backed RBAC + ABAC · distributed locking · circuit breakers and multi-provider LLM fallback

Delivery: Build-once/promote-everywhere immutable images across 4 environments · blue/green weighted canary with automatic rollback · version-as-code promotion by pull request · GitHub OIDC (no long-lived credentials)

Stack: Python 3.11, TypeScript, Next.js 16 / React 19, AWS (Lambda, API Gateway v2, EventBridge, SQS, ECR, ALB, KMS, Secrets Manager, VPC/PrivateLink), MongoDB Atlas, PostgreSQL, Redis, Terraform, Docker, GitHub Actions, SonarQube

Quality: pylint 10.00/10, mypy strict, ≥90% coverage, SonarQube quality gates, Checkov/Trivy/TFLint on all infrastructure

Sethi Enterprise (Smart Expert Targeting for HCP Insight) — unified KOL discovery and qualitative-research engine for pharmaceutical teams

Python 3.11 · FastAPI/Starlette (async) · Pydantic v2 · PostgreSQL + SQLAlchemy 2.0 async (asyncpg) · MongoDB (async PyMongo) · Redis / RedisCluster · AWS S3 · Pinecone + Cohere rerank · LangGraph/LangChain · OpenAI · Anthropic · Gemini · Perplexity · AWS Textract · Microsoft Graph (MSAL) · React 18 + TypeScript · Vite · TanStack Query · shadcn/Radix · Tailwind · Docker Compose · nginx · Jenkins · Terraform · AWS Lambda + API Gateway HTTP API v2

- Architected a polyglot-persistence research platform carrying KOL discovery, advisory projects and the full qualitative-research workflow — screening, scheduling, interview, transcript, AI summary, consultation report — over 27 API router modules and a per-domain worker layer, deliberately splitting the datastores by the shape of the data rather than by convenience: PostgreSQL for the relational identity/RBAC core, MongoDB for the expert directory (~18.5k KOL documents) and the stage-based project workflow, Redis for sessions/cache/OTP/rate-limiting, S3 for regulated artifacts (screeners, transcripts, recordings) and Pinecone for semantic retrieval — with the cross-store join (a soft string user reference) made an explicit, documented application-layer contract instead of an accidental one
- Designed a three-attribute RBAC model in which effective permission is role grants ∪ per-user grants ∖ per-user revokes, roles carry tier, data scope and capability scope, and the permission checker is a pure membership test against the token's claims with no superadmin special case — privilege comes from what a role is seeded with, not from a branch in the gate, so there is no bypass path to audit; the effective set is baked into the JWT at login with the Redis session as the revocation layer, and a role edit invalidates affected sessions so the next login re-mints the token
- Specified cross-tenant denial as HTTP 404, never 403, as a tenancy invariant across the multi-org model (host organization plus client pharma organizations): a 403 confirms the record exists and a 404 does not, so the server never leaks that another organization's data exists — a rule enforced at the guard layer rather than left to each endpoint's judgement
- Established audit as middleware, not as a call each handler remembers to make: every request is intercepted, tagged against an 80-plus entry route map into section, action and entity type, attributed to the caller decoded from the JWT, sanitized of secrets, and written to an append-only audit store — best-effort by design, so a transient datastore outage degrades the audit trail rather than failing the user's request, with the same reasoning applied inversely to rate limiting, which fails open on cache outage
- Built the AI subsystem as a self-contained sub-package with its own router, schema and provider layer, orchestrated by a LangGraph state machine compiled once per process (intent classification, then vector / internet / chat / metadata routing, then history persistence), with per-capability provider selection and explicit fallback chains across four model vendors — so a provider outage degrades one capability instead of the product, and a model swap is a service-layer change rather than an application-wide one
- Drove the strangler-fig migration of the monolith to serverless: a versioned, wave-sequenced registry of 124 endpoints drives per-endpoint Lambda scaffolding behind an API Gateway HTTP API v2 with a single centralized Lambda authorizer, so authentication and authorization are implemented once rather than 124 times, and cold-start secret resolution falls back to plain environment variables as a safe no-op — letting the whole surface be validated and tested locally with no AWS credentials
- Resolved a duplicated-account topology by consolidating the serverless plane onto the existing VPC, where two independently applied Terraform roots share exactly one thing — three published network outputs, read one-directionally via remote state — eliminating a redundant VPC and NAT gateway and, more importantly, giving Lambda cold starts VPC endpoints to Secrets Manager instead of a path over the public internet; the cost of that sharing (a plane boundary now resting on security groups rather than on separate VPCs) is documented as a consequence rather than discovered later

Kyrios — service-oriented platform for verified healthcare professionals

Python 3.11 (async) · FastAPI · React 19 · MongoDB · Redis Cluster · Anthropic Claude API · AWS (S3, CloudFront) · Docker · Nginx · Terraform

- Architected a service-oriented platform for verified healthcare professionals — professional community, AI clinical evidence Q&A, automated news ingestion, survey marketplace, rewards wallet and an internal ops console — applying the strangler-fig pattern to retire two legacy systems (MEAN survey-operations platform, Laravel member panel) onto a modern async stack

- Designed the backend as a layered modular monolith — thin routers, domain workers, a single data-access layer — combining the repository pattern with type-level read/write segregation and a facade write seam through which every identity and wallet mutation flows, making auditability and append-only semantics structural rather than conventional
- Engineered an event-driven real-time layer: Redis pub/sub fan-out delivering Server-Sent Events across worker processes, a distributed-mutex pattern (Redis SET NX EX) so scheduled jobs never double-run under horizontal scale, and stateless 12-factor services with session state externalised to Redis Cluster
- Designed a convergent ingestion pipeline on a webhook-primary, scheduled-poll-backstop model, with idempotent writes guaranteed by URL canonicalisation and fuzzy title de-duplication so both paths can fire without producing duplicates, and per-source failure isolation with circuit-breaker-style health flagging
- Authored the AI architecture for a regulated clinical context: versioned provenance contracts carried through to exported PDF metadata, structurally enforced citations via mandatory tool calls, PHI redaction ahead of persistence, prompt-cache splitting, externalised versioned prompts, and asynchronous fire-and-forget dispatch so no user request blocks on inference
- Designed the security and compliance model: tiered membership authorisation, dual-audience session isolation, RBAC with role grants plus per-user overrides under cache-aside invalidation, an append-only consent ledger using revocation-as-new-record, hierarchical legal-document version resolution, cross-cutting audit logging via middleware interception, and a feature-flag kill-switch placing every third-party integration behind a default-off gate

RXLABS — **Lead Full-Stack Engineer / Cloud Architect** **Feb 2025 – Mar 2026**

Owned full-stack development and cloud deployment across 6 projects, 3 running concurrently — API design and frontend through to infrastructure and CI/CD.

Centralised Observability & Analytics Platform

Python · AWS (Kinesis, Glue, Redshift, Athena, Lambda, EventBridge, S3, CloudWatch, DMS, Macie, KMS, GuardDuty) · Prometheus · Grafana · CloudFormation

- Built an event-driven pipeline consolidating real-time logs and metrics from all 6 production applications into a single queryable Redshift warehouse
- Developed an Athena analytics layer over the S3 data lake, enabling SQL-driven incident investigation and audit queries across every environment
- Integrated Prometheus metrics collection and built Grafana dashboards giving unified observability across all services, replacing manual log trawling with live monitoring
- Configured DMS for cross-environment replication, keeping staging and production consistent for reliable analytics
- Implemented KMS encryption and Macie automated sensitive-data classification for compliance across the data lake
- Built CloudWatch alerting and EventBridge event-response workflows, cutting mean time to incident detection across production workloads
- Defined the entire estate as parameterised CloudFormation for repeatable, auditable deployments

Subscription Management Platform

FastAPI · React · PostgreSQL · AWS (ECS, RDS, ElastiCache) · Celery · RabbitMQ · Jenkins · Terraform

- Built the FastAPI backend on Repository, Unit of Work, and Factory patterns — clean separation of concerns, fully testable and maintainable at scale
- Architected a multi-cart system carrying 2 recurring plans and 3 one-time purchase types, with intelligent product categorisation and dynamic pricing through a Solidgate integration
- Built the full subscription lifecycle: retry logic, refund workflows, cancellation flows, email notifications, alerts, and audit logs
- Designed the async processing architecture on Celery and RabbitMQ, queuing high-volume orders reliably

- Delivered the platform from scratch against a two-week deadline, directly opening a new revenue stream — now processing 1,000+ orders/day
- Provisioned the AWS infrastructure (ECS, RDS, ElastiCache, EC2 subnets) and automated releases through a Jenkins and Terraform CI/CD pipeline

UMA Learning Platform — subscription mobile and web learning

FastAPI · Primer · Celeris · RevenueCat · PostgreSQL · Selenium · pytest

- Built a rebilling engine from scratch on Primer and Celeris, increasing daily payment authorization rates 40–60%
- Re-architected the product from single-lesson to multi-lesson support, enabling horizontal scalability and an expanded catalogue
- Rebuilt the authentication system from scratch, resolving a critical bug that blocked 20% of active users from logging in
- Ran the ETL extracting and loading user lesson data from CSVs into a normalised multi-table PostgreSQL schema
- Built the admin panel that cut customer-support response time, and implemented onboarding quiz logic for new customers
- Wrote pytest unit and integration tests plus Selenium end-to-end coverage across rebilling, auth, and lesson modules, blocking production regressions

E-Commerce Orchestration Platform

FastAPI · MedusaJS · Next.js · React · PostgreSQL · AWS (ALB, ECS, RDS, ElastiCache)

- Designed the FastAPI backend on Repository, Unit of Work, and Factory patterns, orchestrating between a MedusaJS storefront and a multi-PSP payment layer (Stripe, Netvalve, Solidgate, Primer, OrderGroove)
- Built multi-PSP payment orchestration enabling flexible provider switching and subscription billing with zero downtime
- Led the migration from a legacy admin panel to a standalone MedusaJS platform, improving backend control and developer velocity
- Built a separate FastAPI backend and Next.js doctor portal, unifying patient-facing and clinical workflows on one architecture
- Automated an Intercom-to-Viber customer-support integration, removing manual handoffs
- Deployed on multi-AZ AWS — ALB, ECS, RDS, and ElastiCache Redis failover inside private subnets for low-latency cross-service communication

AI-Powered Medical Records Platform (EMR)

Django · React · Claude API · Payload CMS · ETL · GCP (Cloud LB, Cloud SQL HA, Memorystore, Cloud Armor)

- Architected the ETL pipeline for automated PDF medical-record ingestion, LLM-powered biomarker extraction, and structured report generation
- Integrated the Claude API to parse unstructured PDF records into queryable patient data, eliminating manual data entry for doctors
- Built a no-code report builder on Django and Payload CMS, letting non-technical admins customise report templates without engineering support
- Built the React frontend and the Django REST API endpoints behind the admin panel and patient-facing features
- Executed a Shopify-to-EMR data migration, mapping and transforming e-commerce records into structured medical record formats
- Deployed on a GCP high-availability stack: Cloud Load Balancer, managed instance groups across 2 zones, Cloud SQL HA, Memorystore Redis, and a Cloud Armor WAF

Real-Time Doctor Monitoring System

FastAPI · React · WebSocket · Google Workspace API · Slack API · Ansible · EC2

- Built a full-stack monitoring application integrating Google Meet and third-party call platforms to track doctor attendance, punctuality, and call duration in real time

- Developed the WebSocket backend delivering instant push notifications to a React dashboard and Slack alerts, removing manual monitoring entirely
- Built the KPI analytics dashboard surfacing attendance patterns, punctuality rates, and performance metrics for management
- Automated previously manual booking workflows, achieving a ~90% reduction in scheduling overhead
- Improved call management and customer-handling processes, reducing support escalations across the platform
- Provisioned and configured EC2 infrastructure with Ansible for consistent, repeatable deployments

CXUNIFY — **Team Lead, Data Engineering & Full-Stack** **2023 – 2024**

Led full-stack and data platform development for an MSP SaaS analytics product.

FastAPI · PostgreSQL · PySpark · Databricks · TypeScript · Azure (Container Apps, Data Factory, Synapse, Data Lake Gen2, Event Hub) · Terraform · Azure DevOps

- Designed and implemented a Medallion architecture (Bronze / Silver / Gold) on Azure Data Lake Gen2 — raw ingestion in Bronze, cleansed and conformed data in Silver, aggregated business-ready datasets in Gold for Synapse consumption
- Built the end-to-end ETL platform on PySpark, Databricks, and Azure Data Factory, moving MSP operational data through all three Medallion layers into Synapse for near-live reporting
- Architected the Azure Landing Zone to Microsoft CAF best practice — hub-and-spoke VNet topology, network segmentation, Azure Firewall, policy-as-code governance, and role-based access control enforced through Terraform
- Structured all infrastructure as reusable, versioned Terraform modules (networking, compute, storage, security) managed through Azure DevOps pipelines for consistent, auditable deployments across dev, staging, and production
- Implemented real-time event streaming with Azure Event Hub, feeding telemetry into the Medallion pipeline for near-live MSP reporting dashboards
- Migrated the legacy monolith to independently scalable, containerised FastAPI microservices deployed via Azure Container Apps
- Wrote and optimised complex SQL queries and stored procedures, eliminating data bugs and significantly improving retrieval performance
- Built an API testing framework simulating physical MSP devices, eliminating hardware procurement costs for the QA team entirely
- Deployed the full Azure observability stack (Monitor, Log Analytics, Application Insights) with Microsoft Sentinel for SIEM, and managed access through Azure AD RBAC
- Mentored junior engineers on full-stack patterns, data engineering practice, and IaC workflows

VISIBILITY — **Team Lead, Full-Stack Development** **2020 – 2023**

Employee of the Year. Led the complete overhaul of an MSP monitoring platform — frontend, backend, and deployment pipeline rebuilt from the ground up, collecting telemetry at 10-second intervals.

Django · React · Angular · Node.js · MySQL · Jenkins · AWS (EC2, ALB, RDS, CloudFront, S3, SES, SQS) · Terraform · Ansible

- Migrated the backend from Node.js to Django and rebuilt the client-facing application in React, resolving persistent responsiveness issues and eliminating performance bottlenecks
- Replaced a standalone Angular admin dashboard with a Django-based admin panel, eliminating an entire server and reducing infrastructure overhead
- Designed and implemented the cell-based (stamp) architecture for per-client infrastructure isolation — each client receiving a dedicated, independently managed stack: CloudFront to S3, an SES/SQS mail queue, and an ALB across EC2 collectors writing to RDS
- Abstracted that pattern into parameterised Terraform modules, so a new client environment is provisioned in minutes from a single variable file with no manual infrastructure work

- Rewrote and optimised the complex MySQL stored procedures holding core business logic, eliminating client-side hanging and significantly improving query response times
- Built a Jenkins CI/CD pipeline automating multi-server EC2 deployments, eliminating manual releases and saving the team hours weekly
- Implemented a comprehensive unit and integration testing suite, reducing regression time and improving code stability
- Trained and mentored new developers and QA engineers, and wrote documentation covering the majority of platform features
- **Business impact:** grew the customer base from 1 to 10 clients and the team from 3 to 16, and cut infrastructure costs 67% ($4,800/yr) through right-sizing and IaC adoption

CODEV — **Full-Stack Engineer** **2019 – 2020**

- Built a fundraising platform (Flask API, React, PostgreSQL) covering donation processing, campaign management, and the RESTful API layer
- Developed an EMR platform for healthcare data management (Flask API, React, MySQL)
- Deployed containerised microservices on AWS EKS, and built RESTful APIs and admin modules for patient and clinic management including appointment booking

EARLY CAREER

SKYLUSTER — Full-Stack Engineer *(2018 – 2019)* · Built multiple crowdfunding platforms (React, Vue.js, Django, Node.js) with payment processing, plus a crypto template marketplace shipping each buyer a storefront, admin, and dedicated provisioned server.

HYUZAN — Full-Stack Engineer *(2017 – 2018)* · Built a cryptocurrency exchange blog and CMS (Django REST Framework, Vue.js, React) with a real-time chart pipeline on Tornado WebSockets, and Bash-automated on-premise server deployment.

WIDEOUT — Frontend Engineer *(2016 – 2017)* · Produced interactive HTML5 games and advertisements for clients including Disney and Vikings, and built the frontend template library that set the team's coding standards.

FREELANCE — Junior Developer *(2015 – 2016)* · Built C++ games with SDL2/SFML, a Java Spring Boot REST application, and Python web-scraping tools with Beautiful Soup.

EDUCATION & CERTIFICATIONS

ATENEO DE ZAMBOANGA UNIVERSITY — **BS Information Technology** **2010 – 2015**

AWS Certified Solutions Architect – Professional (SAP-C02) — *in progress, expected 2026*

REFERENCES

Frankie Alvarado — CEO at Flare Kite and Visibility One · frankie.alvarado@outlook.com

Ace Quimpo — Head of Engineering at Anervea · linkedin.com/in/ace-aubrey-q-03077b53

Mac Ignacio — Director of Engineering at Anervea · linkedin.com/in/macignacio

Paul — Tech Lead at RxLabs · paul@andyou.ph

Ardy — Product Manager at RxLabs · vrillveloso@gmail.com